

# Intro to Rust

A study guide for people who already ship in  
Python and PowerShell

---

ELI5 → Tech 101 → Deeper

Mapped to how you already think: scripts, ops tooling,  
CyClaw (Python), and CyClaw-Net-Viewer (Rust).

For Christopher Grady · September 2026

## Contents

1. How to use this guide
2. ELI5 — What Rust is (and isn't)
3. Rosetta stone — Python / PowerShell / Rust
4. Tech 101 — The core ideas you'll hit first
5. Deeper — Ownership, types, errors, async, Cargo
6. Your stack bridge — toward Net-Viewer / systems work
7. Practice path (2-week sketch)
8. Cheat sheet & next reads

# 1. How to use this guide

You're not a blank slate. You already automate with **PowerShell** (Veeam/SCCM/ops DNA) and build real systems in **Python** (CyClaw: FastAPI, graphs, config, installers). You're also touching **Rust** via CyClaw-Net-Viewer. This guide translates Rust into those habits instead of pretending you're a CS freshman.

**Path:** skim ELI5 once → live in Tech 101 with tiny exercises → use Deeper as a reference when the compiler yells. When stuck, ask: "What would I do in Python?" then "What did PowerShell force me to be explicit about?" Rust usually wants the second answer.

FACT: Rust's learning curve is front-loaded (compiler fights). INFERENCE: that fight is the product — it moves whole classes of bugs from production into compile time, which matches why you care about invariants and fail-closed design in CyClaw.

## 2. ELI5 — What Rust is (and isn't)

Imagine three kitchen roles:

- **PowerShell** — a power drill + tape measure for the house (Windows/ops). Great for tasks: install, schedule, CredMan, one-liners.
- **Python** — a full workshop. Fast to build machines (agents, APIs, data). Sometimes the machine rattles if you forget a bolt (None, race, wrong type).
- **Rust** — a workshop that won't let you leave until every bolt is torqued. Slower to start; the finished tool is harder to break accidentally.

Rust is a **systems language**: close to the metal like C/C++, but with a compiler that tracks **who owns memory** and **who's allowed to borrow it**. No garbage collector like Python. No "just mutate everything" like a lot of PowerShell/.NET scripts.

### ELI5 one-liners

- **Ownership** — every value has one owner. When the owner leaves the room, the value is cleaned up. No mystery leaks; no use-after-free.
- **Borrowing** — you can lend a value (read-only many times, or write-only once). Same rule as "one person edits the doc; many can view."
- **Result / Option** — errors and missing values are types, not vibes. Closer to PowerShell's careful `$ErrorActionPreference` + checking exits than to Python's "maybe raise, maybe None."
- **Cargo** — npm/pip/uv for Rust: build, test, deps, lockfile. Your friend.

Rust is **not**: a replacement for every Python script; a magic money printer; "C but chill." It **is**: great for CLIs, network tools, performance-sensitive local agents, and anything where a crash mid-flight is expensive (hello Net-Viewer / egress watching).

## 3. Rosetta stone — Python / PowerShell / Rust

| Idea             | Python                               | PowerShell                        | Rust                                              |
|------------------|--------------------------------------|-----------------------------------|---------------------------------------------------|
| Run a file       | <code>python app.py</code>           | <code>.\script.ps1</code>         | <code>cargo run</code>                            |
| Deps             | <code>pip / uv / requirements</code> | <code>modules / gallery</code>    | <code>Cargo.toml</code> + <code>Cargo.lock</code> |
| Package / module | <code>package / import</code>        | <code>module / .psm1</code>       | <code>mod / use / crates</code>                   |
| Null / missing   | <code>None</code>                    | <code>\$null</code>               | <code>Option&lt;T&gt;</code> (Some / None)        |
| Errors           | <code>raise / try</code>             | <code>throw / -ErrorAction</code> | <code>Result&lt;T,E&gt; + ?</code>                |

|                 |                  |                          |                            |
|-----------------|------------------|--------------------------|----------------------------|
| List            | list             | @()                      | Vec<T>                     |
| Dict / map      | dict             | hashtable @{}            | HashMap<K,V>               |
| String          | str (unicode)    | [string]                 | String vs &str             |
| Truthy checks   | if x:            | if (\$x)                 | explicit bool / matches    |
| Loop file lines | for line in f    | Get-Content   %          | BufRead::lines()           |
| HTTP client     | httpx / requests | Invoke-RestMethod        | reqwest (crate)            |
| Async           | asyncio / await  | jobs / Runspaces         | async/.await + tokio       |
| Types           | optional hints   | [int] casts              | required, inferred locally |
| OOP-ish         | classes          | classes / PSCustomObject | structs + impl + traits    |

Mental model: PowerShell taught you **explicit pipelines and exit codes**. Python taught you **ship fast and iterate**. Rust asks you to keep the explicitness and pay up front so production doesn't pay later.

## 4. Tech 101 — Core ideas you'll hit first

### 4.1 Hello + Cargo

Like a Python project with pyproject + venv, or a PowerShell module folder with a manifest — Cargo owns the project.

```
cargo new hello_rust
cd hello_rust
cargo run          # build + run
cargo test         # like pytest
cargo build --release # optimized binary
```

Cargo.toml ≈ requirements.txt + setup metadata. Cargo.lock ≈ lockfile (commit it for apps).

### 4.2 Variables, mutability, types

Python: everything rebindable. PowerShell: variables are mutable by default. Rust: **immutable by default** — you opt into mutation with mut.

```
let x = 5;           // immutable binding
let mut y = 5;       // can reassign y
y = 6;
let name = "cgfixit"; // &str string slice
let owned = String::from("cgfixit"); // owned String
```

Python str is always owned/GC'd. In Rust you constantly choose: **borrowed view** (&str) vs **owned buffer** (String). Same idea as “pass by reference vs copy,” but enforced.

### 4.3 Functions + control flow

```
fn add(a: i32, b: i32) -> i32 {
    a + b // last expression = return (no semicolon)
}

fn describe(n: i32) -> &'static str {
    if n > 0 { "pos" } else if n < 0 { "neg" } else { "zero" }
}

match n {
    0 => println!("zero"),
    1..=10 => println!("small"),
    _ => println!("other"),
}
```

match is like a PowerShell switch that must cover every case (exhaustiveness). Python match (3.10+) is the cousin; Rust's version is stricter and everywhere.

### 4.4 Structs, impl, traits (not classes)

Python classes mix data + methods + inheritance. PowerShell has classes too, but ops life is often hashtables + functions. Rust splits: **data** (struct) + **methods** (impl) + **shared behavior** (traits ≈ interfaces / protocols).

```
struct Endpoint {
    addr: String,
    port: u16,
}

impl Endpoint {
```

```

    fn new(addr: String, port: u16) -> Self {
        Self { addr, port }
    }
    fn label(&self) -> String {
        format!("{}", self.addr, self.port)
    }
}

trait Describe {
    fn describe(&self) -> String;
}
impl Describe for Endpoint {
    fn describe(&self) -> String { self.label() }
}

```

`&self` = borrow self read-only (like a method that doesn't mutate). `&mut self` = exclusive write. `self` = take ownership (rare; consumes the value).

## 4.5 Option and Result (stop pretending null is fine)

Python: None and exceptions. PowerShell: \$null and terminating/non-terminating errors. Rust: make absence and failure **visible in the type**.

```

fn find_port(name: &str) -> Option<u16> {
    if name == "gate" { Some(8787) } else { None }
}

fn load_key() -> Result<String, std::io::Error> {
    std::fs::read_to_string("/home/box/.CyClaw/dogfood.env")
}

// ? propagates Err upward (like rethrow), unwraps Ok
fn boot() -> Result<(), std::io::Error> {
    let _key = load_key()?;
    Ok(())
}

```

Prefer `? +` meaningful errors over `.unwrap()` in real code. `unwrap` is fine in experiments and tests — it's raise with less ceremony.

## 4.6 Ownership in one page (Tech 101 version)

```

let a = String::from("CyClaw");
let b = a;           // move: a is INVALID now
// println!("{a}"); // compile error
println!("{b}");

let c = b.clone();    // deep copy when you truly need two owners
let r = &c;           // shared borrow
println!("{r} {c}"); // both ok – read borrows can overlap

```

Python would just bump a refcount. PowerShell/.NET would GC. Rust forces you to see the move. That's the whole game for the first month.

## 5. Deeper — when you're ready to go further

### 5.1 Borrow checker rules (formal-ish)

- At any time: either one `&mut T` OR any number of `&T`, never both.
- References must not outlive the data (lifetimes). Often inferred; sometimes you write `'a`.
- Moves invalidate the old name for non-Copy types (String, Vec, most structs).
- `i32`, `bool`, etc. are Copy — assignment duplicates bits, old name still works.

When the compiler complains, it's usually: you mutated while a read borrow lived; you returned a reference to something that died; or you used a value after moving it. Translate the error to English before “fixing randomly.”

### 5.2 Lifetimes (only as deep as you need)

A lifetime is a name for “how long this reference is valid.” You mostly meet them in function signatures that return references tied to inputs:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

Read `'a` as: “output lives as long as both inputs.” If this makes your eyes glaze, skip — write `owned String` returns first, optimize to borrows later (same as premature optimization in Python).

### 5.3 Enums as data modeling superpower

Python: Union types / dataclasses / exceptions. PowerShell: sparse. Rust enums with payloads replace a lot of subclass trees:

```
enum ConnState {
    Idle,
    Open { addr: String, port: u16 },
    Failed { reason: String },
}

fn show(s: &ConnState) {
    match s {
        ConnState::Idle => println!("idle"),
        ConnState::Open { addr, port } => println!("{addr}:{port}"),
        ConnState::Failed { reason } => println!("err: {reason}"),
    }
}
```

This is why Rust fits “topology as policy” brains: illegal states become harder to represent.

### 5.4 Traits, generics, dyn

- **Generics** — `fn foo<T: Describe>(x: T)` like Python `TypeVar + Protocol`, but checked.
- **dyn Trait** — trait objects for runtime polymorphism (vtable), closer to Python duck typing at a cost.
- Common traits: `Clone`, `Debug`, `Default`, `Display`, `From/Into`, `Iterator`, `Send`, `Sync`.

Derive early: `#[derive(Debug, Clone)]` on structs — like `auto __repr__` + copy helper.

### 5.5 Error handling in real tools

For CLIs/services you'll want something like `thiserror` / anyhow (ecosystem crates): structured errors vs quick prototyping. Pattern maps to CyClaw's typed error codes more than to bare Python exceptions.

```
// Conceptual pattern (stdlib-only sketch)
fn parse_port(s: &str) -> Result<u16, String> {
    s.parse::<u16>().map_err(|e| format!("bad port: {e}"))
}
```

## 5.6 Collections + iterators

Iterators are lazy pipelines — think PowerShell pipeline or Python generators:

```
let ports = vec![8787, 8790, 22];
let open: Vec<_> = ports.iter()
    .filter(|p| **p > 1024)
    .map(|p| format!("port-{:p}"))
    .collect();
```

## 5.7 Async (tokio) — only when you need it

Python `asyncio` and Rust `async` look similar (`async fn / .await`) but Rust's `async` doesn't mean "green threads free lunch." You pick a runtime (usually **tokio**). For Net-Viewer-style UI + sockets, you'll meet `async` boundaries carefully. Rule: get sync solid first; don't `async` everything like a TikTok tutorial.

## 5.8 Unsafe — the fire extinguisher behind glass

`unsafe` means "I am personally vouching for invariants the compiler can't check." You can write huge useful programs without it. When you need it (FFI, raw sockets edge cases), isolate it like `CyClaw` isolates OOB modules.

## 6. Your stack bridge

Given how you work today:

- **Python / CyClaw** — keep as the agent brain, RAG, policy graph, install orchestration.
- **PowerShell** — keep as Windows install/ops (CredMan, tasks, Veeam DNA).
- **Rust** — local performance + safety: Net-Viewer, parsers, anything that watches the machine closely.

### Practical learning loop tied to Net-Viewer instincts

- Parse structured text (logs, CSV) with ownership-friendly APIs.
- Model connection state as enums, not boolean soup.
- Surface errors to UI/status (you already felt this with Finder reveal spawn UX).
- Add tests with `cargo test`; prefer fail-closed defaults.

Don't rewrite CyClaw in Rust for sport. Use Rust where crashes/races are costly and Python where iteration speed wins — same “sell labor / pick tools” doctrine, applied to languages.

## 7. Practice path (2-week sketch)

### Week 1 — Tech 101 reps

- Day 1–2: `cargo new`; variables; functions; `match`; small CLI that prints args.
- Day 3–4: structs + `impl`; `Vec`/`HashMap`; read a file line by line.
- Day 5–7: `Option`/`Result` everywhere; rewrite a tiny PowerShell one-liner as Rust CLI (e.g. filter listening ports from a text dump).

### Week 2 — Deeper + bridge

- Enums for state machines; traits for shared behavior.
- Add clap for CLI args (like `argparse` / `param blocks`).
- Read Net-Viewer code paths you care about; trace one error from spawn to status bar.
- Write three tests for a pure function; make the compiler your code reviewer.

## 8. Cheat sheet & next reads

### Commands

```
rustup update
cargo new NAME | cargo init
cargo check      # typecheck fast
cargo run / test / fmt / clippy
rustc --explain E0XXX
```

### Mindset checklist when stuck

- Did I move a value and still use the old name?
- Do I have a live & borrow while trying `&mut`?
- Should this return `Option`/`Result` instead of panicking?
- Would an owned `String` make the lifetime issue disappear?

## Reads (in order)

- The Rust Book (official) — chapters 1–11 first.
- Rust By Example — skim patterns.
- Rustlings — drills when you want reps.
- Your own Net-Viewer PRs — best textbook you'll finish.

---

Closing: hard to run with the weight of gold *or* lead — Rust just makes the weight visible at compile time. Use that. Keep Python for speed of thought, PowerShell for Windows ops, Rust for the blades you don't want shattering in your hand.

Generated for Christopher Grady · September 2026 · not a substitute for The Book